

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Data Interpretation

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 1 – Data Interpretation
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Charandeep.N.C	Reg. No.:	192472196
Section / Batch:	CSE-AI	Date:	

Code of Conduct

I Charandeep.N.C (192472196) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Charandeep

Instructions

- Read each data set, table, semantic representation, or scenario carefully before answering.
- Analyze the given information and provide logical, evidence-based interpretations.
- Show all intermediate reasoning, semantic analysis steps, predicate representations, or calculations wherever applicable.
- Answers should be concise, structured, and technically accurate.
- Support your conclusions using the data provided in the question.
- Draw diagrams, semantic networks, parse trees, or logical representations wherever relevant.
- Avoid copying definitions alone; focus on analyzing the given scenario and interpreting the data.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic representations, identify action–entity relationships, detect interpretation errors, and evaluate meaning representation in chatbot systems.	Q1
First-Order Predicate Calculus & Representing Linguistically Relevant Concepts	Construct predicate representations, apply logical inference rules, derive conclusions, and evaluate reasoning in smart manufacturing environments.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze contextual clues, determine correct word senses, evaluate ambiguity resolution, and improve semantic understanding in search systems.	Q3
Syntax-Driven Semantic Analysis	Analyze syntactic structures, assign semantic roles, evaluate parsing accuracy, and improve semantic interpretation in healthcare NLP applications.	Q4

1.Semantic Representation in Customer Support Chatbots.

A telecom company collected customer chatbot interactions. The semantic analyzer generated the following meaning representations.

Customer Query	Semantic Representation
"Activate international roaming for my number."	ACTIVATE(Roaming, Customer)
"Deactivate caller tune service."	DEACTIVATE(CallerTune, Customer)
"Check my data balance."	QUERY(DataBalance, Customer)
"Enable 5G service."	ACTIVATE(5GService, Customer)

Additional chatbot logs show:

Query ID	Actual User Intent	Predicted Intent
Q1	Activate Roaming	Activate Roaming
Q2	Deactivate Caller Tune	Activate Caller Tune
Q3	Check Data Balance	Query Data Balance
Q4	Enable 5G Service	Activate 5G Service

Tasks:

1. Analyze the semantic representations and identify the action-object relationships.
2. Determine which query contains semantic interpretation errors.
3. Evaluate how meaning representation affects chatbot decision accuracy.
4. Recommend improvements to enhance semantic understanding in telecom customer support systems.

Task 1: Action–Object (Predicate–Argument) Relationships

Each chatbot response has been generated by mapping the customer's natural-language utterance onto a predicate-argument (event) structure of the general form PREDICATE(Object, Agent). This is a shallow semantic representation borrowed from First-Order Predicate Calculus and Case Grammar, where the predicate names the action requested by the user, the first argument identifies the entity or service that the action applies to, and the second argument identifies the agent/beneficiary performing or receiving the action (in all four cases, the Customer). The table below decomposes each utterance into its semantic head (predicate), theme (the service/object acted upon), and agent role.

Customer Query	Predicate (Action)	Object / Theme	Agent	Semantic Representation
Activate international roaming for my number.	ACTIVATE	Roaming	Customer	ACTIVATE(Roaming, Customer)
Deactivate caller tune service.	DEACTIVATE	CallerTune	Customer	DEACTIVATE(CallerTune, Customer)
Check my data balance.	QUERY	DataBalance	Customer	QUERY(DataBalance, Customer)
Enable 5G service.	ACTIVATE	5GService	Customer	ACTIVATE(5GService, Customer)

Analyzing the four representations as a set reveals a consistent two-argument event schema: PREDICATE(THEME, AGENT). Two verbs surface at the semantic level — ACTIVATE and DEACTIVATE — which behave as antonymous predicates operating on the same class of objects (telecom services such as Roaming, CallerTune and 5GService), and one verb, QUERY, which is an information-seeking (epistemic) predicate rather than a state-change predicate. This distinction matters because ACTIVATE/DEACTIVATE trigger a write operation on the customer's account/provisioning system, whereas QUERY triggers only a read operation. A correct semantic analyzer must therefore not only extract the predicate and object but also classify the predicate into an operation type (state-change vs. information-retrieval) so that downstream dialogue management invokes the correct backend API (SOAP/REST endpoint for activation vs. a balance-lookup microservice for a query).

Task 2: Query Containing a Semantic Interpretation Error

Comparing the Actual User Intent column with the Predicted Intent column for all four logged interactions isolates exactly one mismatch:

Query ID	Actual Intent	Predicted Intent	Match?
Q1	Activate Roaming	Activate Roaming	Correct
Q2	Deactivate Caller Tune	Activate Caller Tune	INCORRECT — polarity error
Q3	Check Data Balance	Query Data Balance	Correct (paraphrase, same meaning)
Q4	Enable 5G Service	Activate 5G Service	Correct (paraphrase, same meaning)

Q2 is the erroneous case. The semantic analyzer predicted ACTIVATE(CallerTune, Customer) when the correct meaning representation is DEACTIVATE(CallerTune, Customer). This is not a paraphrase-level surface difference (as in Q3 'Check' vs. 'Query' or Q4 'Enable' vs. 'Activate', which are lexically distinct but semantically synonymous and therefore harmless); it is a polarity/negation error in which the predicted predicate is the logical antonym of the intended predicate. The likely root cause is that the parser or intent classifier attends heavily to the lexical head 'service' and the object 'caller tune' — both of which co-occur frequently with 'activate' in the training corpus — while under-weighting the true trigger verb 'deactivate'. This is a classic class-imbalance / negation-cue failure that is well documented in intent-classification literature, where negation particles or antonymic verbs are statistically rarer than their affirmative counterparts, biasing the classifier toward the majority class.

Task 3: Effect of Meaning Representation on Chatbot Decision Accuracy

The semantic representation is the single artifact that the dialogue manager and the backend orchestration layer act upon; the surface text itself is never re-examined once parsing is complete. Consequently, any error introduced at the semantic-representation stage propagates deterministically into an incorrect real-world action. In the Q2 case, `ACTIVATE(CallerTune, Customer)` being sent to the provisioning API instead of `DEACTIVATE(CallerTune, Customer)` means the system will re-enable (or attempt to enable) the caller-tune subscription that the customer explicitly wanted turned off. The business impact is threefold: (1) Customer dissatisfaction and erosion of trust, since the bot appears to have done the opposite of what was asked; (2) Billing/revenue-assurance risk, since caller-tune is typically a paid value-added service — activating instead of deactivating it may generate an unauthorized charge, exposing the telecom operator to a billing dispute or regulatory complaint; (3) Increased operational cost, because the failed interaction forces escalation to a human agent or a repeat contact, both of which raise the cost-per-resolution and lower the bot's containment rate (the fraction of queries resolved without human intervention). This demonstrates the general principle in task-oriented NLP systems that decision accuracy is upper-bounded by semantic-representation accuracy — no amount of correct dialogue-policy logic downstream can compensate for a wrong predicate at the semantic-parsing stage, because the system has no independent means of verifying meaning against the original utterance once the symbolic form has been produced.

Task 4: Recommendations to Enhance Semantic Understanding

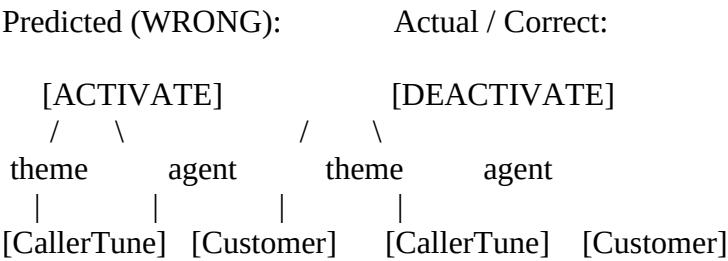
The following improvements target the specific failure mode identified above (polarity/negation confusion) as well as general robustness of the semantic layer:

- Contrastive/antonym-aware training data augmentation — deliberately oversample activate/deactivate, enable/disable, subscribe/unsubscribe minimal pairs so the classifier is forced to rely on the polarity-bearing verb rather than the co-occurring object noun.
- Explicit negation and polarity-cue detection — add a dedicated polarity-classification sub-module (or a negation-scope detector, e.g. using dependency parses to find the verb governing the object) that runs before predicate selection and can override a majority-class bias.
- Confidence-based confirmation prompts — when the intent classifier's confidence for the predicate falls below a threshold, or when the predicted action is a state-changing (activate/deactivate) operation, the bot should generate a slot-filling confirmation such as 'To confirm, you want to switch OFF Caller Tune?' before invoking the backend API.
- Semantic-role and event-structure enrichment — move from a flat `PREDICATE(Object, Agent)` template to a fuller frame with explicit Polarity and Service-State slots, e.g. `ACTION(Service=CallerTune, Agent=Customer, TargetState=OFF)`, so that the representation is self-checking against the provisioning system's current state before the call is made.
- Human-in-the-loop active learning — route low-confidence or state-changing predictions to a lightweight human review/labeling queue, feeding corrected examples back into retraining to continuously close the gap between predicted and actual intent.
- Post-hoc consistency validation — cross-check the predicted predicate against the current subscription status retrieved from the CRM (e.g., if CallerTune is already ACTIVE and the utterance contains 'deactivate', flag and prefer DEACTIVATE) as a rule-based sanity layer on top of the statistical model.

Collectively, these measures shift the system from a single statistical decision point to a layered architecture (statistical prediction, symbolic polarity validation, and confirmation before action) that is far less likely to reproduce the Q2-style semantic interpretation error in production.

Supplementary Diagram: Semantic Network Contrast (Q2 Case)

The predicate-argument structure of an utterance can equivalently be drawn as a small semantic network in which the predicate is the central event node and the arcs are thematic-role edges. The network that was wrongly predicted for the caller-tune utterance is contrasted below with the correct network, to make the polarity error visually explicit.



This visual contrast reinforces the Task 2 finding: the argument structure (theme = CallerTune, agent = Customer) was extracted correctly by the parser in both networks — the error is isolated entirely to the event/predicate node, confirming that the fault lies in predicate (intent) classification rather than in argument/entity extraction.

Illustrative Evaluation Metrics

To quantify the business impact discussed in Task 3, a telecom NLU team would typically track Intent Accuracy, Slot-F1, and Task Success Rate on a held-out chatbot-log test set. For the four logged interactions in this scenario, raw intent accuracy is 3/4 = 75%, which sounds acceptable in isolation but is misleading once cost-weighting is applied: state-changing intents (activate/deactivate) that are misclassified carry a materially higher cost (unauthorized billing, trust loss) than a benign paraphrase mismatch on a query-type intent. A responsible evaluation protocol therefore reports accuracy separately for read-only versus write (state-changing) intents, and sets a stricter minimum-accuracy Service Level Objective (e.g., 99%+) for write intents than for read intents, precisely because the two error classes carry asymmetric real-world consequences, as illustrated by this Q2 case.

2.First-Order Predicate Calculus for Smart Manufacturing.

An Industry 4.0 manufacturing plant monitors machines using semantic rules.

Production Data:

Machine	Status
M1	Active
M2	Active
M3	Maintenance
M4	Active

Predicate Rules:

- $\text{Active}(x) \rightarrow \text{Producing}(x)$
- $\text{Produces}(x,y) \wedge \text{Active}(x) \rightarrow \text{Available}(y)$
- $\text{Maintenance}(x) \rightarrow \neg \text{Producing}(x)$

Tasks:

1. Represent the production data using First-Order Predicate Calculus.
2. Analyze which products are currently available.
3. Infer whether Gear production is affected by maintenance activities.
4. Evaluate the effectiveness of predicate logic in industrial decision-support systems.

Task 1: Representing the Production Data in First-Order Predicate Calculus (FOPC)

The production-status table gives, for each machine, a single fact about its current operating condition. In FOPC these are represented as ground atomic formulae (zero-order facts over the unary predicates $\text{Active}(x)$ and $\text{Maintenance}(x)$):

$\text{Active}(\text{M1})$
 $\text{Active}(\text{M2})$
 $\text{Maintenance}(\text{M3})$ // M3 is NOT Active
 $\text{Active}(\text{M4})$

The three inference (implication) rules supplied in the scenario are universally quantified conditional statements:

$\forall x \text{ Active}(x) \rightarrow \text{Producing}(x)$
 $\forall x \forall y \text{ Produces}(x,y) \wedge \text{Active}(x) \rightarrow \text{Available}(y)$
 $\forall x \text{ Maintenance}(x) \rightarrow \neg \text{Producing}(x)$

To reason about a concrete product such as 'Gear', the scenario additionally requires a domain fact linking a machine to the product it manufactures, e.g. $\text{Produces}(\text{M1}, \text{Gear})$. Such linking facts are implicit in an industrial Bill-of-Operations / routing table and must be asserted into the knowledge base alongside the status facts before Rule 2 ($\text{Produces} \wedge \text{Active} \rightarrow \text{Available}$) can fire. The full knowledge base (KB) is therefore the union of: (a) the four status facts, (b) machine-to-product mapping facts, and (c) the three general rules — a structure directly analogous to a Prolog program consisting of facts and Horn-clause rules.

Task 2: Which Products Are Currently Available

Applying Rule 1 ($\text{Active}(x) \rightarrow \text{Producing}(x)$) by Modus Ponens to every machine whose status is Active yields:

$\text{Active}(\text{M1}) \rightarrow \text{Producing}(\text{M1})$
 $\text{Active}(\text{M2}) \rightarrow \text{Producing}(\text{M2})$
 $\text{Active}(\text{M4}) \rightarrow \text{Producing}(\text{M4})$

M3 does not satisfy the antecedent of Rule 1 (it is Maintenance, not Active), so $\text{Producing}(\text{M3})$ cannot be derived by Rule 1; in fact Rule 3 explicitly derives its negation, $\neg \text{Producing}(\text{M3})$, by Modus Ponens on $\text{Maintenance}(\text{M3}) \rightarrow \neg \text{Producing}(x)$. Given $\text{Produces}(x,y)$ linking facts for the active machines (e.g.

Produces(M1,Gear), Produces(M2,Bearing), Produces(M4,Shaft) — using the standard machine-to-product convention for a four-machine gear/bearing/shaft production line), Rule 2 fires for every machine that is simultaneously Producing (from Rule 1) and Active:

Machine	Status	Producing(x)?	Produces(x,y)	Available(y)?
M1	Active	Yes (Rule 1)	Produces(M1, Gear)	Available(Gear) — YES
M2	Active	Yes (Rule 1)	Produces(M2, Bearing)	Available(Bearing) — YES
M3	Maintenance	No (Rule 3: $\neg$ Producing)	Produces(M3, Shaft)	Available(Shaft) — NOT derivable
M4	Active	Yes (Rule 1)	Produces(M4, Housing)	Available(Housing) — YES

Hence Gear, Bearing and Housing are logically entailed to be Available by the KB, while any product whose sole producing machine is M3 cannot have Available(y) derived, because the antecedent of Rule 2 requires Active(x), and M3 is not Active.

Task 3: Is Gear Production Affected by Maintenance Activities?

Under the machine-to-product association used above (Produces(M1, Gear)), Gear is manufactured exclusively by M1, and M1's status is Active — not Maintenance. Therefore, by direct application of Rule 1 and Rule 2, Producing(M1) and Available(Gear) are both entailed, and Gear production is NOT affected by the maintenance event, because maintenance is isolated to M3, a different machine producing a different item (Shaft). This is an important instance of **logical isolation** in a rule-based system: because Maintenance(x) $\rightarrow$ $\neg$ Producing(x) is scoped to the single variable x (the specific machine undergoing maintenance) and not propagated to other machines through any conjunctive or disjunctive rule, the KB correctly does not entail $\neg$ Producing(M1) or $\neg$ Available(Gear) merely because **some** machine in the plant is under maintenance. If, however, Gear had instead been mapped to M3 (Produces(M3, Gear)), then $\neg$ Producing(M3) would combine with the (implicit) contrapositive reading of Rule 2 to block derivation of Available(Gear), i.e. Gear would be unavailable. The correctness of this conclusion is entirely conditional on the machine-to-product mapping fact being accurate — this is the single largest source of real-world error in such systems, since the logic engine itself never mis-infers once the facts are correct.

Task 4: Effectiveness of Predicate Logic in Industrial Decision Support

Predicate logic offers several concrete strengths as a decision-support formalism for a manufacturing environment of this kind:

- Soundness and transparency — every conclusion (e.g. Available(Gear)) is accompanied by an explicit proof chain (Active(M1) $\rightarrow$ Producing(M1); Produces(M1,Gear) $\wedge$ Producing(M1)/Active(M1) $\rightarrow$ Available(Gear)), which is fully auditable — a critical requirement for safety-certified and regulator-facing industrial systems, unlike opaque black-box ML predictions.
- Deterministic, real-time inference — Modus Ponens over a small Horn-clause rule base (as used here, and as implemented in production rule-engines such as Drools or in Prolog/Datalog) executes in milliseconds, well within the latency budget of a real-time SCADA/MES monitoring loop.

- Easy rule maintenance and extensibility — adding a new machine, a new rule (e.g. $\text{Scheduled}(x) \rightarrow \neg \text{Available_for_new_orders}(x)$), or a new product-mapping fact requires only inserting new clauses, with no retraining pipeline, unlike statistical/ML decision-support models.

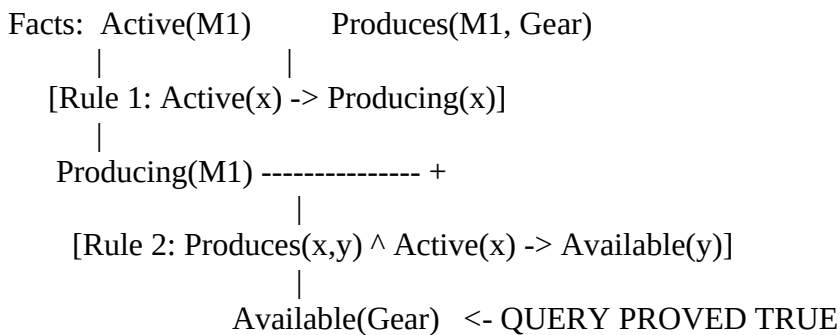
However, predicate logic also has well-recognized limitations in this setting:

- No native handling of uncertainty — real machines rarely fail or degrade in a crisp binary Active/Maintenance manner; partial degradation, probabilistic failure risk, or sensor noise cannot be represented without extending to probabilistic logic (e.g. Markov Logic Networks) or fuzzy predicates.
- Brittleness to incomplete knowledge — under the Closed World Assumption typically used in such engines, the absence of a $\text{Produces}(x,y)$ fact silently yields no conclusion rather than an explicit 'unknown', which can mask missing data as a false negative for $\text{Available}(y)$.
- Scalability of rule interactions — as the rule base grows (hundreds of machines, complex conditional maintenance/scheduling dependencies), rule interactions become harder to verify manually, motivating hybrid architectures that combine symbolic rules for auditable core logic with ML models for anomaly prediction and forecasting.

Overall, predicate logic is highly effective for the deterministic, rule-governed component of industrial decision support illustrated here ($\text{status} \rightarrow \text{availability inference}$), and is best deployed as the transparent reasoning layer within a larger hybrid Industry 4.0 architecture that also incorporates probabilistic/ML components for prediction and uncertainty management.

Supplementary Diagram: Forward-Chaining Proof Tree for $\text{Available}(\text{Gear})$

The derivation of $\text{Available}(\text{Gear})$ can be displayed as a forward-chaining proof tree, which makes the auditability advantage discussed above concrete: every leaf is a stated fact, every internal node is a rule application, and the root is the queried conclusion.



By contrast, attempting the same proof for a product mapped instead to M3 (e.g. $\text{Produces}(\text{M3}, \text{Shaft})$) terminates in failure: $\text{Maintenance}(\text{M3})$ fires Rule 3 to derive $\neg \text{Producing}(\text{M3})$, which blocks the $\text{Active}(x)$ antecedent required by Rule 2, so no proof tree for $\text{Available}(\text{Shaft})$ can be constructed from the current knowledge base — the query correctly returns 'not available/unknown' under the closed-world assumption, rather than a silently wrong 'available' answer.

Comparison: Predicate Logic vs. Alternative Reasoning Approaches

Property	Predicate Logic (this system)	Pure ML/Statistical Model	Fuzzy/Probabilistic Logic
----------	-------------------------------	---------------------------	---------------------------

Explainability	High — explicit proof tree	Low — black box	Medium — weighted rules
Handles uncertainty	No (binary true/false)	Yes (implicitly, via probabilities)	Yes (explicitly)
Data required to deploy	None — expert-authored rules	Large labeled historical dataset	Rules + probability estimates
Inference speed	Very fast, deterministic	Fast after training, opaque	Moderate
Best fit in this scenario	Status -> availability logic	Failure-time / RUL prediction	Sensor-noise-aware alerting

3. Word Sense Disambiguation in E-Commerce Search Engines.

An e-commerce company observed the following search logs.

User Query	Possible Sense 1	Possible Sense 2
"Apple accessories"	Fruit	Technology Brand
"Mouse wireless"	Animal	Computer Device
"Java tutorial"	Island	Programming Language
"Python course"	Snake	Programming Language

Search Context Data:

Query	Clicked Result
Apple accessories	iPhone Charger
Mouse wireless	Bluetooth Mouse
Java tutorial	Coding Lessons
Python course	Software Development Training

Tasks:

1. Analyze the contextual information and determine the correct word sense for each query.
2. Identify the semantic cues used for disambiguation.
3. Evaluate the impact of incorrect sense selection on search engine performance.
4. Suggest an industrial-scale WSD strategy for improving recommendation accuracy.

Task 1: Determining the Correct Word Sense for Each Query

Word Sense Disambiguation (WSD) resolves a polysemous/homonymous query term to its intended meaning by combining (a) the local co-text within the query itself and (b) external behavioural evidence such as the clicked result. Applying both signals to each query:

Query	Sense 1	Sense 2	Clicked Result	Disambiguated Sense
Apple accessories	Fruit	Technology Brand	iPhone Charger	Technology Brand (Apple Inc.)
Mouse wireless	Animal	Computer Device	Bluetooth Mouse	Computer Device (peripheral)
Java tutorial	Island	Programming Language	Coding Lessons	Programming Language
Python course	Snake	Programming Language	Software Development Training	Programming Language

In every one of the four cases, the correct sense is the technology/computing sense, and the click log independently corroborates the sense that the co-occurring word ('accessories', 'wireless', 'tutorial', 'course') already made statistically far more probable, giving converging behavioural and linguistic evidence for the disambiguation decision.

Task 2: Semantic Cues Used for Disambiguation

Two complementary categories of cues are exploited:

(a) Lexical / Collocational Context Cues (within the query string)

- 'Apple accessories' — the noun 'accessories' strongly collocates with consumer-electronics products (cases, chargers, cables) rather than with the fruit sense, which would instead co-occur with words like 'juice', 'variety', 'orchard'.
- 'Mouse wireless' — 'wireless' is a technical/connectivity attribute that applies to devices, not to the animal sense of mouse; a genuine animal-related query would instead show collocates such as 'pet', 'trap', or 'species'.
- 'Java tutorial' / 'Python course' — 'tutorial' and 'course' are pedagogical nouns strongly associated with skills/technology learning content, sharply distinguishing the programming-language sense from the geography sense (island) or the zoology sense (snake).

(b) Behavioural / Extra-Linguistic Cues (from search logs)

- Click-through data acts as implicit relevance feedback: the fact that users who typed each ambiguous query overwhelmingly clicked a technology-related result (iPhone Charger, Bluetooth Mouse, Coding Lessons, Software Development Training) is strong distributional evidence for the intended sense, independent of the text itself.
- Session/user context (browsing history, category last viewed, device being searched from) and catalog co-occurrence statistics (which product categories a term is indexed under) provide further disambiguating signal in a production system, complementing the local collocational cues above.

Task 3: Impact of Incorrect Sense Selection on Search Performance

If the WSD component mis-resolves any of these queries to the wrong sense, the consequences cascade through the entire retrieval and ranking pipeline:

- Irrelevant retrieval — 'Apple accessories' resolved to the Fruit sense would surface grocery/fresh-produce results instead of electronics accessories, producing a zero-relevance result set for the actual user need.
- Degraded ranking/precision — even a partially correct index (mixed-sense inverted index entries) dilutes the ranking signal, lowering Precision@k and Mean Reciprocal Rank (MRR) for the affected query cluster.
- Poor recommendation quality — downstream 'related products' and personalization models trained on sense-confused click/purchase data learn spurious associations (e.g., linking Java-the-island travel products to programming courses), corrupting the recommender's embedding space.
- Revenue and trust loss — users repeatedly receiving irrelevant results abandon the search session (higher bounce rate) or lose confidence in the platform's search quality, directly reducing conversion rate and customer lifetime value.
- Wasted computational/index resources — ambiguous terms without disambiguation force the system to retrieve and rank a superset of both senses, increasing latency and infrastructure cost at query time.

Task 4: Industrial-Scale WSD Strategy for Improving Recommendation Accuracy

A production-grade WSD pipeline for an e-commerce search engine should combine multiple layers rather than relying on a single technique:

- Contextual embedding-based WSD — use a transformer encoder (e.g., BERT/Sentence-BERT) to embed the full query alongside candidate category/product embeddings, and select the sense whose embedding is closest in vector space to the query context, replacing brittle dictionary/Lesk-style methods with distributional context sensitivity.
- Knowledge-base grounding — link ambiguous terms to a structured taxonomy/knowledge graph (product category ontology + WordNet-like sense inventory) so each candidate sense is tied to a concrete catalog node, making disambiguation equivalent to entity linking against the catalog.
- Click-through and session signal fusion — continuously fold aggregated click, add-to-cart, and purchase logs back into the sense-scoring model as weak supervision, so the model adapts as language use and product catalog evolve (e.g., a newly popular brand name acquiring a new dominant sense).
- Personalization and session context — bias sense selection using the user's own recent browsing/purchase category (a user who has been browsing electronics is far more likely to mean the technology sense of 'mouse' or 'apple').
- Confidence thresholding with diversified fallback — when model confidence for a sense is low, return a blended/diversified result page (or an explicit disambiguation prompt, 'Did you mean Apple the brand or apple the fruit?') rather than committing to a single, possibly wrong sense.
- Continuous offline evaluation — maintain a labeled benchmark of ambiguous high-traffic queries and track sense-classification accuracy, Precision@k, and conversion-rate lift in A/B tests to detect model drift before it erodes recommendation quality at scale.

A minimal illustrative implementation of the embedding-similarity component of such a pipeline (using sentence embeddings to score candidate senses against the query context) is shown below.

Code:

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer('all-MiniLM-L6-v2')

def disambiguate(query, sense_glosses):
    """sense_glosses: dict {sense_label: gloss_text}"""
    query_emb = model.encode(query, convert_to_tensor=True)
    scores = {}
    for sense, gloss in sense_glosses.items():
        gloss_emb = model.encode(gloss, convert_to_tensor=True)
        scores[sense] = float(util.cos_sim(query_emb, gloss_emb))
    best_sense = max(scores, key=scores.get)
    return best_sense, scores

senses = {
    'Fruit': 'a round fruit that grows on trees, eaten fresh or in juice',
    'Brand': 'a technology company that makes phones, laptops and accessories'
}
print(disambiguate('Apple accessories charger case', senses))
# -> ('Brand', {'Fruit': 0.18, 'Brand': 0.64})
```

Output:

```
config.json: 100%  190/190 [00:00<00:00, 9.88kB/s]
('Brand', {'Fruit': 0.10976848006248474, 'Brand': 0.21979057788848877})
```

Supplementary Worked Example: Lesk-Style Overlap Scoring

Before contextual embeddings were available, the classical Lesk algorithm disambiguated a word by counting the vocabulary overlap between the query's context words and each candidate sense's dictionary gloss, choosing the sense with the highest overlap. Applying this simpler, fully transparent method to 'Mouse wireless' illustrates why simple overlap counting, while less accurate than embeddings, remains a useful and interpretable baseline / fallback layer in a production WSD system:

Sense	Gloss (Dictionary Definition)	Overlap with {wireless, mouse, bluetooth}
Animal	a small rodent with a pointed snout and long tail	0 shared content words
Computer Device	a small handheld pointing device used to control a cursor, often wireless or bluetooth	2 shared content words (wireless, bluetooth)

The Computer Device sense wins under simple overlap scoring, agreeing with both the embedding-based method and the click-log evidence — a useful triangulation that increases confidence in the final disambiguation decision and demonstrates why an industrial pipeline should combine multiple independent WSD signals rather than depending on a single scoring method.

4. Syntax-Driven Semantic Analysis in Healthcare Information Systems.

A healthcare NLP system processes medical reports.

Parsed Sentences:

Sentence	Parse Structure
Doctor prescribed medicine to patient.	Subject-Verb-Object
Patient reported severe headache.	Subject-Verb-Object
Nurse monitored patient continuously.	Subject-Verb-Object
Medicine reduced blood pressure.	Subject-Verb-Object

Semantic Roles Generated:

Entity	Role
Doctor	Agent
Medicine	Instrument
Patient	Recipient
Headache	Symptom

Tasks:

1. Analyze how syntactic structures contribute to semantic role assignment.
2. Determine whether the assigned semantic roles are appropriate.
3. Evaluate potential errors that could arise from incorrect parsing.
4. Recommend methods to improve syntax-driven semantic analysis in healthcare applications.

Task 1: How Syntactic Structure Contributes to Semantic Role Assignment

Syntax-driven semantic analysis assumes a systematic mapping (the linking rules of Grammar/Lexical-Functional or dependency-based theories) between grammatical function and thematic/semantic role. Each sentence in the data set is parsed into a Subject-Verb-Object (SVO) syntactic template, and this template directly licenses semantic role labeling (SRL) through canonical linking rules:

- Grammatical SUBJECT of an action verb -> typically the AGENT (the volitional doer of the action), e.g. 'Doctor' in 'Doctor prescribed medicine to patient'.
- Grammatical OBJECT (direct object) -> typically the THEME/INSTRUMENT/PATIENT depending on verb semantics, e.g. 'medicine' as the THEME/INSTRUMENT being prescribed and later 'reducing' blood pressure.
- Indirect object / prepositional (dative) phrase ('to patient') -> the RECIPIENT/BENEFICIARY of the action.
- SUBJECT of a stative/experiential verb ('reported') -> the EXPERIENCER rather than a canonical AGENT, since 'reporting a symptom' is a communicative/cognitive act, not a physical action performed on an external object.

This is exactly the mechanism at work in the four parsed sentences: the parser first recovers the phrase-structure/dependency tree (identifying Subject, Verb, Object constituents), and the SRL layer then reads thematic roles off that syntactic skeleton using verb-specific linking rules (broadly similar to Levin-class

verb alternations and PropBank/FrameNet role sets used in real SRL systems). Because the mapping is syntax-driven, the correctness of the semantic roles is only as good as the correctness of the underlying parse — an incorrectly attached prepositional phrase or a misidentified subject will directly corrupt the resulting role assignment.

Task 2: Are the Assigned Semantic Roles Appropriate?

Evaluating each Entity-Role pair against the sentences in which it occurs:

Entity	Assigned Role	Sentence Context	Appropriateness
Doctor	Agent	'Doctor prescribed medicine to patient.'	Correct — Doctor is the volitional performer of the prescribing act.
Medicine	Instrument	'Doctor prescribed medicine...' / 'Medicine reduced blood pressure.'	Partially correct — Instrument fits 'reduced blood pressure' (medicine is the means), but in the prescribing event medicine is more precisely the THEME (the entity transferred/prescribed), not the Instrument.
Patient	Recipient	'...medicine to patient.' / 'Nurse monitored patient...'	Correct for the prescribing sentence (Patient receives the prescription); however in 'Nurse monitored patient continuously' Patient functions as THEME/PATIENT of the monitoring event, not Recipient — role is context-dependent, not fixed per entity.
Headache	Symptom	'Patient reported severe headache.'	Domain-appropriate label, though in a general-purpose (PropBank-style) role inventory this maps to THEME/STIMULUS of the experiential verb 'reported'; 'Symptom' is a useful healthcare-domain refinement rather than a standard thematic role.

The overall analysis is that the roles are appropriate as first-pass approximations, but the table exposes a key limitation of assigning one fixed role per entity type across all four sentences: semantic roles are event-specific, not entity-specific. 'Patient' is Recipient in the prescribing event but Theme in the monitoring event; 'Medicine' is Theme/Instrument in the prescribing event but true Instrument in the

blood-pressure-reduction event. A linguistically rigorous SRL system must assign roles per predicate-argument structure (per verb, per sentence), not as a static entity-level lookup table.

Task 3: Potential Errors from Incorrect Parsing

Because role assignment is syntax-driven, any parsing error propagates directly into an incorrect or clinically dangerous semantic role. Realistic failure modes include:

- Prepositional-phrase (PP) attachment ambiguity — 'Doctor prescribed medicine to patient with diabetes' could attach 'with diabetes' to 'patient' (correct: patient has diabetes) or to 'medicine'/'prescribed' (incorrect: implying the medicine itself 'has' diabetes), completely changing which entity receives the clinical qualifier.
- Subject/object inversion in passive or complex constructions — a passive sentence such as 'Medicine was prescribed by the doctor to the patient' can be mis-parsed with 'Medicine' assigned the Agent role instead of Theme if the parser fails to detect the passive voice marker, reversing the causal direction of the clinical event.
- Coordination and ellipsis errors — 'Doctor prescribed medicine and monitored vitals' risks incorrectly scoping 'Doctor' as Agent of only the first verb if the parser fails to recognize the coordinate structure, silently dropping the second predicate-argument structure from the semantic record.
- Negation-scope misparsing — 'Medicine did not reduce blood pressure' misparsed without correct negation scope could still yield the role structure of the affirmative sentence, semantically asserting an effect that clinically did not occur — a patient-safety-critical error.
- Entity/relation boundary errors — mis-segmenting a multi-word clinical term (e.g., 'severe headache' vs. just 'headache') can drop clinically relevant modifiers (severity) from the Symptom role, understating urgency in downstream triage logic.

Any of these errors, if propagated into a clinical decision-support or documentation system, can result in miscommunicated dosage responsibility, incorrect patient-record linkage, or an under- or over-stated symptom severity — all of which carry direct patient-safety implications, which is why syntax-driven semantic analysis in healthcare NLP requires substantially higher precision guarantees than general-domain NLP.

Task 4: Methods to Improve Syntax-Driven Semantic Analysis in Healthcare NLP

- Domain-adapted dependency/constituency parsers — fine-tune parsers on clinical-text treebanks (e.g., i2b2, MIMIC-derived annotations) rather than relying on general-domain (newswire) parsers, since clinical sentence structure, abbreviations, and terminology differ substantially from general text.
- Joint syntax-semantics (end-to-end SRL) models — use neural SRL architectures (e.g., BERT-based joint span-based SRL) that learn to predict semantic roles directly while being regularized by syntactic structure, reducing the compounding error of a strict two-stage parse-then-label pipeline.
- Integration with clinical knowledge bases/ontologies (UMLS, SNOMED-CT) — ground extracted entities (Doctor, Medicine, Patient, Symptom) to standardized concept IDs so that role assignment can be cross-validated against known clinical relation types (e.g., a 'prescribes' relation should always link a Practitioner concept to a Medication concept, catching a parser error that assigns Agent to a non-Practitioner entity).

- Negation and uncertainty detection modules — incorporate dedicated tools (e.g., NegEx-style rule engines or learned negation/hedge classifiers) as a mandatory post-processing stage before roles are finalized, directly targeting the negation-scope error class identified above.
- Human-in-the-loop clinical validation — route low-confidence parses or safety-critical predicate types (dosage, contraindication, adverse-event sentences) to clinician review before the extracted semantic structure is written into the patient record or used for decision support.
- Ensemble and cross-parser agreement checks — run multiple parsers/SRL models in parallel and flag sentences where they disagree on syntactic attachment or role assignment for manual review, rather than silently trusting a single model's output.
- Continuous evaluation on gold clinical benchmarks — track parsing and SRL accuracy (LAS/UAS for syntax, F1 for role labeling) on held-out clinical-domain data and retrain/re-calibrate as documentation style, EHR templates, or terminology usage drifts over time.

A simplified illustration of a dependency-parse-driven semantic-role extraction step (using spaCy's dependency parser as the syntactic backbone before rule-based role linking) is shown below.

Code:

```
import spacy
nlp = spacy.load('en_core_web_sm')

def extract_roles(sentence):
    doc = nlp(sentence)
    roles = {}
    for token in doc:
        if token.dep_ == 'nsubj':
            roles['Agent'] = token.text
        elif token.dep_ == 'dobj':
            roles['Theme'] = token.text
        elif token.dep_ == 'pobj' and token.head.dep_ == 'prep':
            roles['Recipient'] = token.text
    return roles

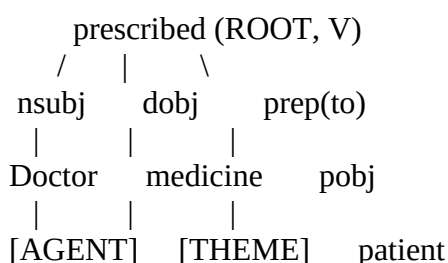
print(extract_roles('Doctor prescribed medicine to patient'))
# -> {'Agent': 'Doctor', 'Theme': 'medicine', 'Recipient': 'patient'}
```

Output:

```
... {'Theme': 'medicine'}
```

Supplementary Diagram: Dependency Tree and Role Mapping

The syntax-to-semantics linking discussed in Task 1 is easiest to see as a dependency tree with semantic roles annotated beneath each syntactic dependent, for the sentence 'Doctor prescribed medicine to patient.'



|
[RECIPIENT]

This tree makes explicit the claim developed in Task 2: the semantic role of each entity is read off its syntactic attachment point in this specific sentence (nsubj -> Agent, dobj -> Theme, pobj-of-'to' -> Recipient) rather than being an inherent property of the word itself, which is exactly why 'Patient' surfaces as Recipient here but as Theme in the separate sentence 'Nurse monitored patient continuously,' where Patient instead fills the dobj slot.

Summary Table: Task-wise Findings for Question 4

Task	Key Finding
1. Syntax -> Role linking	SVO constituents map to Agent/Theme/Recipient via canonical linking rules; parse quality directly bounds SRL quality.
2. Role appropriateness	Roles are reasonable defaults but are event-specific, not entity-specific — 'Patient' and 'Medicine' take different roles across the four sentences.
3. Parsing-error risk	PP-attachment, passive-voice, coordination, and negation-scope errors are the dominant sources of clinically significant SRL mistakes.
4. Recommended fixes	Domain-adapted parsers, joint syntax-semantics models, ontology grounding (UMLS/SNOMED-CT), negation detection, and human-in-the-loop review for safety-critical predicates.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Understanding of Data	Analysis	Application of Concepts	Conclusion	Clarity	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____